

基本级别之外调整性能需要一些对处理器微体系结构的理解,描述处理器用来实现它的指令集体结构的底层机制。对于乱序处理器的情况,只需要知道一些关于操作、容量、延迟和功能单元发射时间的信息,就能够基本地预测程序的性能了。

我们研究了一系列技术,包括循环展开、创建多个累积变量和重新结合,它们可以利用现代处理器提供的指令级并行。随着对优化的深入,研究产生的汇编代码以及试着理解机器如何执行计算变得重要起来。确认由程序中的数据相关决定的关键路径,尤其是循环的不同迭代之间的数据相关,会收获良多。我们还可以根据必须要计算的操作数量以及执行这些操作的功能单元的数量和发射时间,计算一个计算的吞吐量界限。

包含条件分支或与内存系统复杂交互的程序,比我们最开始考虑的简单循环程序,更难以分析和优化。基本策略是使分支更容易预测,或者使它们很容易用条件数据传送来实现。我们还必须注意存储和加载操作。将数值保存在局部变量中,使得它们可以存放在寄存器中,这会很有帮助。

当处理大型程序时,将注意力集中在最耗时的部分变得很重要。代码剖析程序和相关的工具能帮助我们系统地评价和改进程序性能。我们描述了 GPROF,一个标准的 Unix 剖析工具。还有更加复杂完善的剖析程序可用,例如 Intel 的 VTUNE 程序开发系统,还有 Linux 系统基本上都有的 VALGRIND。这些工具可以在过程级分解执行时间,估计程序每个基本块(basic block)的性能。(基本块是内部没有控制转移的指令序列,因此基本块总是整个被执行的。)

参考文献说明

我们的关注点是从程序员的角度描述代码优化,展示如何使书写的代码能够使编译器更容易地产生高效的代码。Chellappa、Franchetti 和 Püschel 的扩展的论文[19]采用了类似的方法,但关于处理器的特性描述得更详细。

有许多著作从编译器的角度描述了代码优化,形式化描述了编辑器可以产生更有效代码的方法。Muchnick 的著作被认为是最全面的[80]。Wadleigh 和 Crawford 的关于软件优化的著作[115]覆盖了一些我们已经谈到的内容,不过它还描述了在并行机器上获得高性能的过程。Mahlke 等人的一篇比较早期的论文[75],描述了几种为编译器开发的将程序映射到并行机器上的技术,它们是如何能够被改造成利用现代处理器的指令级并行的。这篇论文覆盖了我们讲过的代码变换,包括循环展开、多个累积变量(他们称之为累积变量扩展(accumulator variable expansion))和重新结合(他们称之为树高度减少(tree height reduction))。

我们对乱序处理器的操作的描述相当简单和抽象。可以在高级计算机体系结构教科书中找到对通用原则更完整的描述,例如 Hennessy 和 Patterson 的著作[46, 第2~3章]。Shen 和 Lipasti 的书[100]提供了对现代处理器设计深入的论述。

家庭作业

- 5.13 假设我们想编写一个计算两个向量 u 和 v 内积的过程。这个函数的一个抽象版本对整数和浮点数类型,在 x86-64 上 CPE 等于 14~18。通过进行与我们将抽象程序 `combine1` 变换为更有效的 `combine4` 相同类型的变换,我们得到如下代码:

```
1  /* Inner product. Accumulate in temporary */
2  void inner4(vec_ptr u, vec_ptr v, data_t *dest)
3  {
4      long i;
5      long length = vec_length(u);
6      data_t *udata = get_vec_start(u);
7      data_t *vdata = get_vec_start(v);
8      data_t sum = (data_t) 0;
9
10     for (i = 0; i < length; i++) {
11         sum = sum + udata[i] * vdata[i];
12     }
13     *dest = sum;
14 }
```

测试显示,对于整数这个函数的 CPE 等于 1.50,对于浮点数据 CPE 等于 3.00。对于数据类型 `double`,内循环的 x86-64 汇编代码如下所示:

```

Inner loop of inner4. data_t = double, OP = *
udata in %rbp, vdata in %rax, sum in %xmm0
i in %rcx, limit in %rbx

1  .L15:                                loop:
2  vmovsd 0(%rbp,%rcx,8), %xmm1         Get udata[i]
3  vmulsd (%rax,%rcx,8), %xmm1, %xmm1   Multiply by vdata[i]
4  vaddsd %xmm1, %xmm0, %xmm0          Add to sum
5  addq $1, %rcx                       Increment i
6  cmpq %rbx, %rcx                    Compare i:limit
7  jne .L15                            If !=, goto loop

```

假设功能单元的特性如图 5-12 所示。

- A. 按照图 5-13 和图 5-14 的风格,画出这个指令序列会如何被译码成操作,并给出它们之间的数据相关如何形成一条操作的关键路径。
 - B. 对于数据类型 `double`,这条关键路径决定的 CPE 的下界是什么?
 - C. 假设对于整数代码也有类似的指令序列,对于整数数据的关键路径决定的 CPE 的下界是什么?
 - D. 请解释虽然乘法操作需要 5 个时钟周期,但是为什么两个浮点版本的 CPE 都是 3.00。
- * 5.14 编写习题 5.13 中描述的内积过程的一个版本,使用 6×1 循环展开。对于 x86-64,我们对这个展开的版本的测试得到,对整数数据 CPE 为 1.07,而对两种浮点数据 CPE 仍然为 3.01。
- A. 解释为什么在 Intel Core i7 Haswell 上运行的任何(标量)版本的内积过程都不能达到比 1.00 更小的 CPE 了。
 - B. 解释为什么对浮点数据的性能不会通过循环展开而得到提高。
- * 5.15 编写习题 5.13 中描述的内积过程的一个版本,使用 6×6 循环展开。对于 x86-64,我们对这个函数的测试得到对整数数据的 CPE 为 1.06,对浮点数据的 CPE 为 1.01。
- 什么因素制约了性能达到 CPE 等于 1.00?
- * 5.16 编写习题 5.13 中描述的内积过程的一个版本,使用 $6 \times 1a$ 循环展开产生更高的并行性。我们对这个函数的测试得到对整数数据的 CPE 为 1.10,对浮点数据的 CPE 为 1.05。
- ** 5.17 库函数 `memset` 的原型如下:

```
void *memset(void *s, int c, size_t n);
```

这个函数将从 `s` 开始的 `n` 个字节的内存区域都填充为 `c` 的低位字节。例如,通过将参数 `c` 设置为 0,可以用这个函数来对一个内存区域清零,不过用其他值也是可以的。

下面是 `memset` 最直接的实现:

```

1  /* Basic implementation of memset */
2  void *basic_memset(void *s, int c, size_t n)
3  {
4      size_t cnt = 0;
5      unsigned char *schar = s;
6      while (cnt < n) {
7          *schar++ = (unsigned char) c;
8          cnt++;
9      }
10     return s;
11 }

```

实现该函数一个更有效的版本,使用数据类型为 `unsigned long` 的字来装下 8 个 `c`,然后用字级的写遍历目标内存区域。你可能发现增加额外的循环展开会有所帮助。在我们的参考机上,能够把 CPE 从直接实现的 1.00 降低到 0.127。即,程序每个周期可以写 8 个字节。

这里是一些额外的指导原则。在此,假设 `K` 表示你运行程序的机器上的 `sizeof(unsigned long)` 的值。

- 你不可以调用任何库函数。
- 你的代码应该对任意 n 的值都能工作，包括当它不是 K 的倍数的时候。你可以用类似于使用循环展开时完成最后几次迭代的方法做到这一点。
- 你写的代码应该无论 K 的值是多少，都能够正确编译和运行。使用操作 `sizeof` 来做到这一点。
- 在某些机器上，未对齐的写可能比对齐的写慢很多。（在某些非 x86 机器上，未对齐的写甚至可能会导致段错误。）写出这样的代码，开始时直到目的地址是 K 的倍数时，使用字节级的写，然后进行字级的写，（如果需要）最后采用用字节的写。
- 注意 `cnt` 足够小以至于一些循环上界变成负数的情况。对于涉及 `sizeof` 运算符的表达式，可以用无符号运算来执行测试。（参见 2.2.8 节和家庭作业 2.72。）

****5.18** 在练习题 5.5 和 5.6 中我们考虑了多项式求值的任务，既有直接求值，也有用 Horner 方法求值。试着用我们讲过的优化技术写出这个函数更快的版本，这些技术包括循环展开、并行累积和重新结合。你会发现有很多不同的方法可以将 Horner 方法和直接求值与这些优化技术混合起来。理想状况下，你能达到的 CPE 应该接近于你的机器的吞吐量界限。我们的最佳版本在参考机上能使 CPE 达到 1.07。

****5.19** 在练习题 5.12 中，我们能够把前置和计算的 CPE 减少到 3.00，这是由该机器上浮点加法的延迟决定的。简单的循环展开没有改进什么。

使用循环展开和重新结合的组合，写出求前置和的代码，能够得到一个小于你机器上浮点加法延迟的 CPE。要达到这个目标，实际上需要增加执行的加法次数。例如，我们使用 2 次循环展开的版本每次迭代需要 3 个加法，而使用 4 次循环展开的版本需要 5 个。在参考机上，我们的最佳实现能达到 CPE 为 1.67。

确定你的机器的吞吐量和延迟界限是如何限制前置和操作所能达到的最小 CPE 的。

练习题答案

5.1 这个问题说明了内存别名使用的某些细微的影响。

正如下面加了注释的代码所示，结果会将 `xp` 处的值设置为 0：

```
4      *xp = *xp + *xp; /* 2x */
5      *xp = *xp - *xp; /* 2x-2x = 0 */
6      *xp = *xp - *xp; /* 0-0 = 0 */
```

这个示例说明我们关于程序行为的直觉往往会是错误的。我们自然会认为 `xp` 和 `yp` 是不同的情况，却忽略了它们相等的可能性。错误通常源自程序员没想到的情况。

5.2 这个问题说明了 CPE 和绝对性能之间的关系。可以用初等代数解决这个问题。我们发现对于 $n \leq 2$ ，版本 1 最快。对于 $3 \leq n \leq 7$ ，版本 2 最快，而对于 $n \geq 8$ ，版本 3 最快。

5.3 这是个简单的练习，但是认识到一个 `for` 循环的 4 个语句（初始化、测试、更新和循环体）执行的次数是不同的很重要。

代码	min	max	incr	square
A.	1	91	90	90
B.	91	1	90	90
C.	1	1	90	90

5.4 这段汇编代码展示了 GCC 发现的一个很聪明的优化机会。要更好地理解代码优化的细微之处，仔细研究这段代码是很值得的。

A. 在没经过优化的代码中，寄存器 `%xmm0` 简单地被用作临时值，每次循环迭代中都会设置和使用。在经过更多优化的代码中，它被使用的方式更像 `combine4` 中的变量 `x`，累积向量元素的乘积。不过，与 `combine4` 的区别在于每次迭代第二条 `vmovsd` 指令都会更新位置 `dest`。我们可以看到，这个优化过的版本运行起来很像下面的 C 代码：